习题九　排序

一、单项选择题

1．下列内部排序算法中：

A．快速排序　　B.直接插入排序

C．二路归并排序　　D．简单选择排序

E．起泡排序　　F．堆排序

（1） 其比较次数与序列初态无关的算法是（　　）

（2）不稳定的排序算法是（　　）

（3）在初始序列已基本有序（除去 n 个元素中的某 k 个元素后即呈有序，k<<n）的情况下，排序效率最高的算法是（　　）

（4）排序的平均时间复杂度为 O(n•logn)的算法是（　　）为 O(n•n)的算法是（　　）

2．比较次数与排序的初始状态无关的排序方法是(　　)。

A．直接插入排序　　B．起泡排序　　C．快速排序　　D．简单选择排序

3．对一组数据（84，47，25，15，21）排序，数据的排列次序在排序的过程中的变化为

（1） 84 47 25 15 21　（2） 15 47 25 84 21

（3） 15 21 25 84 47　（4） 15 21 25 47 84

则采用的排序是 (　　)。

A．选择　　B．冒泡　　C．快速　　D．插入

4．下列排序算法中(　　)排序在一趟结束后不一定能选出一个元素放在其最终位置上。

A．选择　　B．冒泡　　C．归并　　D．堆

5．一组记录的关键码为（46，79，56，38，40，84），则利用快速排序的方法，以第一个记录为基准得到的一次划分结果为（　　）。

A．(38,40,46,56,79,84)　　B．(40,38,46,79,56,84)

C．(40,38,46,56,79,84)　　D．(40,38,46,84,56,79)

6．下列排序算法中，在待排序数据已有序时，花费时间反而最多的是(　　)排序。

A．冒泡　B．希尔　C．快速　D．堆

7．就平均性能而言，目前最好的内排序方法是(　　)排序法。

A．冒泡　B．希尔插入　C．交换　D．快速

8．下列排序算法中，占用辅助空间最多的是：(　　)

A．归并排序　　B．快速排序　　C．希尔排序　　D．堆排序

9．若用冒泡排序方法对序列{10,14,26,29,41,52}从大到小排序，需进行（　　）次比较。

A．3　　B．10　　C．15　　D．25

10．快速排序方法在（　　）情况下最不利于发挥其长处。

A．要排序的数据量太大　　B．要排序的数据中含有多个相同值

C．要排序的数据个数为奇数　D．要排序的数据已基本有序

11．下列四个序列中，哪一个是堆（　　）。

A．75,65,30,15,25,45,20,10　　B．75,65,45,10,30,25,20,15

C．75,45,65,30,15,25,20,10　　D．75,45,65,10,25,30,20,15

12．有一组数据(15，9，7，8，20，-1，7，4)，用堆排序的筛选方法建立的初始堆为（　　）

A．-1，4，8，9，20，7，15，7　　B．-1，7，15，7，4，8，20，9

C．-1，4，7，8，20，15，7，9　　D．A，B，C 均不对。

二、填空题

1.若待排序的序列中存在多个记录具有相同的键值，经过排序，这些记录的相对次序仍然保持不变，则称这种排序方法是_____的，否则称为_____的。

2.按照排序过程涉及的存储设备的不同，排序可分为_____排序和_____排序。

3．直接插入排序用监视哨的作用是_____。

4.对 n 个记录的表 r[1..n]进行简单选择排序，所需进行的关键字间的比较次数为_____。

5.下面的 c 函数实现对链表 head 进行选择排序的算法,排序完毕,链表中的结点按结点值从小到大链接。请在空框处填上适当内容,每个空框只填一个语句或一个表达式：

```
#include <stdio.h>
typedef struct node {char data; struct node *link; }node;
node *select(node *head)
{node *p,*q,*r,*s;
 p=(node *)malloc(sizeof(node));
 p->link=head; head=p;
 while(p->link!=null)
   {q=p->link; r=p;
    while ((1)_____)
       { if (q->link->data<r->link->data) r=q;
         q=q->link;
       }
    if ((2)_____) {s=r->link; r->link=s->link;
                   s->link= ((3)_____); ((4)_____);}
    ((5)_____) ;
   }
 p=head; head=head->link; free(p); return(head);
}
```

6．下面的排序算法的思想是：第一趟比较将最小的元素放在 r[1]中，最大的元素放在 r[n]中，第二趟比较将次小的放在 r[2]中，将次大的放在 r[n-1]中，…,依次下去，直到待排序列为递增序。（注：<-->）代表两个变量的数据交换）。

```
void  sort(SqList &r,int n)  {
   i=1;
while((1)_____)   {
min=max=1;
for (j=i+1;(2)_____ ;++j)
 {if((3)_____) min=j; else if(r[j].key>r[max].key)  max=j; }
if((4)_____) r[min] < ---- >r[j];
if(max!=n-i+1){if ((5)_____) r[min] < ---- > r[n-i+1]; else ((6)_____); }
i++;
}
```

```
}//sort
```

7．下列算法为奇偶交换排序，思路如下：第一趟对所有奇数的 i,将 a[i]和 a[i+1]进行比较,第二趟对所有偶数的 i,将 a[i]和 a[i+1]进行比较,每次比较时若 a[i]>a[i+1],将二者交换;以后重复上述二趟过程,直至整个数组有序。

```
void oesort (int a[n])
{int flag,i,t;
 do {flag=0;
    for(i=1;i<n;i++,i++)
      if(a[i]>a[i+1])
        {flag=(1)_____; t=a[i+1]; a[i+1]=a[i]; (2)_____;}
    for (3)_____
      if (a[i]>a[i+1])
        {flag=(4)_____;t=a[i+1]; a[i+1]=a[i]; a[i]=t;}
  }while (5)_____;
}
```

三、应用题

1．对于给定的一组键值：83，40，63，13，84，35，96，57，39，79，61，15，分别画出应用直接插入排序、直接选择排序、快速排序、堆排序、归并排序对上述序列进行排序中各趟的结果。

2．判断下列序列是否是堆（可以是小堆，也可以是大堆，若不是堆，请将它们调整为堆）。

（1）100，85，98，77，80，60，82，40，20，10，66

（2）100，98，85，82，80，77，66，60，40，20，10

（3）100，85，40，77，80，60，66，98，82，10，20

（4）10，20，40，60，66，77，80， 82，85，98，100

3．填空并回答相关问题

(1)下面是将任意序列调整为最大堆（MAX HEAP）的算法，请将空白部分填上：

将任意序列调整为最大堆通过不断调用 adjust 函数，即：FOR（i=n/2；i >0；i- -）adjust（list，i，n）；其中 list 为待调整序列所在数组（从下标 1 开始），n 为序列元素个数，adjust 函数为：

```
void  adjust（int list[]，int root，int n）
/*将以 root 为下标的对应元素作为待调整堆的根，待调整元素放在 list 数组中，最大
  元素下标为 n*/
  {int child,rootkey;
    rootkey=list[root];
    child=2*root;
    while(child<=n)
     {if((child<n)&&(list[child]<list[child+1]))
        (1)_____;
      if(rootkey>list[child])
        break;
```

```
        else{List[(2)        ]=list[child];
             child*=2;
             }
        }
        list[child/2]=rootkey;
    }
```

(2). 判断下列序列能否构成最大堆：(12，70，33，65，24，56，48，92，86，33)；
若不能按上述算法将其调整为堆，调整后的结果为：
(　　)。

四、算法设计题：

1. 设计一个用链表表示的直接选择排序算法。

2. 冒泡排序算法是把大的元素向上移（气泡的上浮），也可以把小的元素向下移（气泡的下沉）请给出上浮和下沉过程交替进行的冒泡排序算法（即双向冒泡排序法）。

3. 输入 50 个学生的记录（每个学生的记录包括学号和成绩），组成记录数组，然后按成绩由高到低的次序输出（每行 10 个记录）。排序方法采用选择排序。

4. 已知（k_1, k_2……，k_n）是堆，试写一个算法将（k_1, k_2，……，k_n, k_{n+1}）调整为堆。按此思想写一个从空堆开始一个一个填入元素的建堆算法（题示：增加一个 k_{n+1} 后应从叶子向根的方向调整）。

第九章　排序

一、单项选择题

1.（1） DC　　（2）ADF　　（3）B　　（4）ACF　　BDE

2．D

3．A

4．C

5．C

6．C

7．D

8．A

9．C

10．D

11．C

12．C

二、填空题

1.稳定、不稳定

2.内部、外部

3．免去查找过程中每一步都要检测整个表是否查找完毕，提高了查找效率。

4．n(n-1)/2

5．题中为操作方便，先增加头结点（最后删除），p 指向无序区的前一记录，r 指向最小值结点的前驱，一趟排序结束，无序区第一个记录与 r 所指结点的后继交换指针。

(1)q->link!=NULL (2)r!=p　　(3)p->link (4)p->link=s　　(5)p=p->link

6．.(1)i<n-i+1　(2)j<=n-i+1　(3)r[j].key<r[min].key　(4)min!=i　(5)max==i (6)r[max]<-->r[n-i+1]

7．(1)1　　(2)a[i]=t　　(3)(i=2;i<=n;i+=2)　　(4)1　　(5)flag

三、应用题

1．　解答：①直接插入排序

序号	1	2	3	4	5	6	7	8	9	10	11	12
关键字	83	40	63	13	84	35	96	57	39	79	61	15
i = 2	40	83	[63	13	84	35	96	57	39	79	61	15]
i = 3	40	63	83	[13	84	35	96	57	39	79	61	15]
i = 4	13	40	63	83	[84	35	96	57	39	79	61	15]
i = 5	13	40	63	83	84	[35	96	57	39	79	61	15]
i = 6	13	35	40	63	83	84	[96	57	39	79	61	15]
i = 7	13	35	40	63	83	84	96	[57	39	79	61	15]
i = 8	13	35	40	57	63	83	84	96	[39	79	61	15]
i = 9	13	35	39	40	57	63	83	84	96	[79	61	15]
i = 10	13	35	39	40	57	63	79	83	84	96	[61	15]
i = 11	13	35	39	40	57	61	63	79	83	84	96	[15]
i = 12	13	15	35	39	40	57	61	63	79	83	84	96

②直接选择排序

序号　1　2　3　4　5　6　7　8　9　10　11　12

关键字　83　40　63　13　84　35　96　57　39　79　61　15

i = 1　13　[40　63　83　84　35　96　57　39　79　61　15]

i = 2　13　15　[63　83　84　35　96　57　39　79　61　40]

i = 3　13　15　35　[83　84　63　96　57　39　79　61　40]

i = 4　13　15　35　39　[84　63　96　57　83　79　61　40]

i = 5　13　15　35　39　40　[63　96　57　83　79　61　84]

i = 6　13　15　35　39　40　57　[96　63　83　79　61　84]

i = 7　13　15　35　39　40　57　61　[63　83　79　96　84]

i = 8　13　15　35　39　40　57　61　63　[83　79　96　84]

i = 9　13　15　35　39　40　57　61　63　79　[83　96　84]

i = 10　13　15　35　39　40　57　61　63　79　83　[96　84]

i = 11　13　15　35　39　40　57　61　63　79　83　84　[96]

③快速排序

关键字　83　40　63　13　84　35　96　57　39　79　61　15

第一趟排序后　[15　40　63　13　61　35　79　57　39]　83　[96　84]

第二趟排序后　[13]　15　[63　40　61　35　79　57　39]　83　84　[96]

第三趟排序后　13　15　[39　40　61　35　57]　63　[79]　83　84　96

第四趟排序后　13　15　[35]　39　[61　40　57]　63　79　83　84　96

第五趟排序后　13　15　35　39　[57　40]　61　63　79　83　84　96

第六趟排序后　13　15　35　39　40　[57]　61　63　79　83　84　96

第七趟排序后　13　15　35　39　40　57　61　63　79　83　84　96

④堆排序

关键字：　83　40　63　13　84　35　96　57　39　79　61　15

排序成功的序列：96　84　83　79　63　61　57　40　39　35　15　13

排序过程如图简答题 8－1.1、8－1.2、8－1.3 所示。

⑤归并排序

关键字　83　40　63　13　84　35　96　57　39　79　61　15

第一趟排序后　[40　83]　[13　63]　[35　84]　[57　96]　[39　79]　[15　61]

第二趟排序后　[13　40　63　83]　[35　57　84　96]　[15　39　61　79]

第三趟排序后　[13　35　40　57　63　83　84　96]　[15　39　61　79]

第四趟排序后　13　15　35　39　40　57　61　63　79　83　84　96

2．(1)是大堆；　(2)是大堆；（4）是小堆；

(3)不是堆，调成大堆　100，98，66，85，80，60，40，77，82，10，20

3．(1)①child=child+1；②child/2

(2)不能，调为大堆：92,86,56,70,33,33,48,65,12,24

四、算法设计题：

1．解答：分析：每趟从单链表头部开始，顺序查找当前链值最小的结点。找到后，插入到当前的有序表区的最后。

```
Void selesort ( lklist L )              /* 设链表 L 带头结点 */
```

```
        {  q=L;                          /* 指向第一数据前趋 */
          while ( q-> next !=NULL )
             {  pl = q -> next ;
                minp =pl;              /* minp 指向当前已知的最小数 */
                while ( pl -> next != NULL )
                   { if ( pl -> next -> data < minp -> data )
                      minp = pl -> next ; /* 找到了更小数 */
                      pl = pl -> next ;   /* 继续往下找 */
                    }
                 if ( minp != q -> next)  /* 将最小数交换到第一个位置上 */
                    { r1 = minp -> next ;
                      minp -> next = r1 -> next ;        /* 删除最小数 */
                      r2 = q -> next ;
                      q -> next = r2 -> next ; /* 删除当前表中第一个数 */
                      r1 -> next = q -> next ;
                      q -> next = r1 ;        /* 将最小数插入到第一个位置上 */
                      r2 -> next = minp -> next ;
                      minp -> next = r2 ;     /* 将原第一个数放到最小数原位置上
   */
                     }
                  q = q > next ;              /* 选择下一个最小数 */
              }
    }
```

2. **void** BubbleSort2(**int** a[],**int** n) //相邻两趟向相反方向起泡的冒泡排序算法

```
  { change=1;low=0;high=n-1; //冒泡的上下界
    while(low<high && change)
    { change=0;              //设不发生交换
      for(i=low;i<high;i++)  //从上向下起泡
        if(a[i]>a[i+1]){a[i]<-->a[i+1];change=1;} //有交换，修改标志 change
      high--; //修改上界
      for(i=high;i>low;i--) //从下向上起泡
        if(a[i]<a[i-1]){a[i]<-->a[i-1];change=1;}
      low++; //修改下界
     }//while
  }//BubbleSort2
```

[算法讨论]题目中“向上移”理解为向序列的右端，而“向下移”按向序列的左端来处理。

3. **typedef struct**

```
    { int num; float score; }RecType;
void SelectSort(RecType R[51], int n)
{ for(i=1; i<n; i++)
    { //选择第 i 大的记录，并交换到位
```

```
        k=i;        //假定第 i 个元素的关键字最大
        for(j=i+1;j<=n;j++)      //找最大元素的下标
          if(R[j].score>R[k].score)  k=j;
        if(i!=k)  R[i] <-->R[k];  //与第 i 个记录交换
      }//for
    for(i=1; i<=n; i++) //输出成绩
      { printf("%d,%f",R[i].num,R[i].score); if(i%10==0) printf("\n");}
  }//SelectSort
```

4. 解答：分析：此问题分为两个算法，第一个算法 shift 假设 a[1] , a[2],…,a[k] 为堆，增加一个无素 a[k + 1],把数组 a[1],a[2], …,a[k + 1]调整为堆。第二个算法 heep 从 1 开始调用算法 sift 将整个数组调整为堆。

```
    Void sift (datatype A[ n ] , int  K)              /* n > = k + 1 */
        {  x = A[ K+1] ;
           i = K +1 ;
           while ( ( i/2 > 0 )&&( A[i/2]>x) ) { A[i]= A[i./2]; i = i/2;} /*从下往上插入位置 */
           A[i] = x ;
        }
    Void heap ( datatype A[ n ] ) ;  /* 从 1 开始调用算法 sift , 将整个数组调整为堆 */
  { for ( k = 1 ; k <= n-1; k++ ) sift ( A,k ) ; }
```